

Robofox-human interaction with CV and deep learning

1st Evgenii Shlomov
*Institute of Robotics and
Computer Vision
Innopolis University
Innopolis, Russia
e.shlomov@innopolis.university*

2nd Artyom Tuzov
*Institute of Robotics and
Computer Vision
Innopolis University
Innopolis, Russia
a.tuzov@innopolis.university*

3rd Dmitriy Vizitei
*Institute of Robotics and
Computer Vision
Innopolis University
Innopolis, Russia
d.vizitei@innopolis.university*

Abstract—This project presents an educational case study on deploying a modern deep learning-based human pose estimation system on an embedded GPU platform and integrating it with a quadruped robot for vision-based human–robot interaction. We implement real-time human pose detection on an NVIDIA Jetson Orin Nano 4 GB module mounted on a Unitree A1 robot dog and stream the resulting skeletal keypoints to a separate gesture recognition module developed by a partner team. Our system uses an off-the-shelf YOLO11m-pose model, containerized for ROS 2 Humble and CUDA 12.6 on JetPack 6, and operates at approximately 15 FPS with about 100 ms end-to-end latency at 640×384 resolution when all components of the pipeline are enabled. While we do not introduce novel learning methods, the project systematically addresses practical challenges of running state-of-the-art computer vision models on constrained hardware, abstracting camera sources, and interfacing perception with legacy high-level robot control APIs.

Code repository: https://github.com/Innopolis-Robotics-Society/project_fabian

Demo video: https://drive.google.com/file/d/12ArWhjUcO1zQnqRNFdvkZq8UcSSWfNpf/view?usp=drive_link

I. INTRODUCTION

Real-time human pose estimation has become a key enabler for natural human–robot interaction (HRI) in service robotics, social robots, and interactive demonstrators. Recent real-time object detection and pose estimation architectures such as YOLO [3], transformer-based detectors [1], [2], and one-stage pose models like YOLO-Pose [4] are now mature enough to run on embedded platforms with limited power budgets, including NVIDIA Jetson boards [5]–[7]. However, deploying these models on real robots still requires non-trivial engineering efforts: dealing with legacy SDKs, integrating with ROS, containerizing the software stack, and operating under real-time and safety constraints.

In this project, we focus on the perception layer of a vision-based HRI system for a Unitree A1 quadruped robot. Our primary goal is *not* to propose a new model, but to build and evaluate a robust engineering pipeline that performs real-time human pose detection on an autonomous embedded platform and exposes skeletal data to an external gesture recognition module. The gesture module and high-level mapping from

gestures to robot behaviors are implemented by a partner team; our contribution is the on-board computer vision front-end.

Concretely, we deploy an off-the-shelf YOLO11m-pose model on an NVIDIA Jetson Orin Nano 4 GB module mounted on the robot. The Jetson runs Ubuntu 22.04 with JetPack 6 and CUDA 12.6 [15], [16], with our perception stack encapsulated in Docker containers derived from Ultralytics’ JetPack 6 images. We use a standard USB webcam connected to the Jetson as the primary RGB camera source; although a RealSense depth camera is physically integrated into the robot, its legacy SDK and software stack made it impractical to use within the time frame of the course. The detected 2D body keypoints (17 joints) and associated confidence scores are published as ROS 2 messages and consumed by the gesture recognition node, which in turn commands the robot using a combination of the official Unitree high-level APIs and a UDP/LCM-based communication bridge.

The work is conducted as part of an educational robotics course. The main learning objectives are: (i) to understand how modern deep learning models can be executed on resource-constrained embedded GPUs, (ii) to integrate such models into a ROS 2-based system, and (iii) to interface the perception pipeline with a real quadruped robot using existing SDKs and communication patterns. From a research standpoint, our contributions are modest; from an engineering and educational perspective, the project provides a realistic end-to-end exercise in building an applied, real-time computer vision system on a legged robot.

The rest of the paper is structured as follows. Section II summarizes related work on real-time detection and pose estimation on embedded devices and prior vision-based demonstrations on Unitree A1. Section III describes the hardware and software platform, the perception pipeline, and the ROS 2 integration. Section IV reports basic performance measurements. Section V discusses limitations and lessons learned. Section VI summarizes the project information required for the course submission, and Section VII concludes with directions for future work.

II. RELATED WORK

A. Real-time object detection and pose estimation

The original YOLO (You Only Look Once) framework [3] formulates object detection as a single-stage regression problem from image pixels to bounding boxes and class probabilities, enabling real-time inference (up to 155 FPS for Fast YOLO on a Titan X GPU) while maintaining competitive accuracy compared to multi-stage methods such as R-CNN. Subsequent YOLO variants and transformer-based detectors improved both accuracy and efficiency. Zhao et al. introduce RT-DETR, a real-time DETR-style detector that removes the need for non-maximum suppression and matches YOLO-like performance [1], while Ouyang proposes DEYOv3, which combines a YOLO-style backbone with a DETR-like decoder and a two-stage training scheme to train efficient end-to-end detectors without ImageNet pretraining [2].

For human pose estimation, top-down methods typically apply a person detector followed by a single-person pose estimator, achieving high accuracy at the cost of computation proportional to the number of people. Bottom-up methods, such as OpenPose, first detect all keypoints and then group them into skeletons using associative cues, which offers more stable runtime but introduces complex grouping steps. YOLO-Pose [4] represents a third line: it extends a YOLOv5 detector to directly regress keypoints for each detected person and optimizes an Object Keypoint Similarity (OKS) loss in an end-to-end fashion. This design combines constant runtime with simple post-processing and achieves strong results on COCO, motivating our choice of a YOLO-style pose model for embedded deployment.

B. Embedded deployment and edge benchmarks

Running deep detectors on embedded devices such as Jetson boards has been investigated in several works. Afifi et al. present a robust real-time pedestrian detection framework on Jetson TX2, combining YOLOv3 with coarse-to-fine detection strategies and temporal/spatial heuristics to improve robustness while maintaining real-time performance [5]. Feng et al. systematically benchmark YOLOv3/YOLOv3-tiny/YOLOv4/YOLOv4-tiny on Jetson Nano, Jetson Xavier NX, and Raspberry Pi 4B with Intel NCS2, analyzing FPS, memory, CPU usage, and energy consumption and highlighting the importance of model complexity and hardware choice for edge deployment [6]. Schrijvers et al. deploy a TensorRT-optimized YOLOv3 detector on Jetson TX2 combined with an optical-flow tracker to analyze shopping behaviour in retail stores, achieving around 10 FPS with 81.6% AP on pedestrians [7].

Our work aligns with this line of research in that we also target NVIDIA Jetson hardware, but we focus on a more recent Orin Nano platform, use a newer YOLO11m-pose model, and integrate the perception pipeline into a ROS 2-based quadruped robot system.

C. Unitree A1 and vision-based interaction

Unitree A1 is a widely used quadruped robot featuring an onboard motion controller and a software stack based on the Unitree Legged SDK [10], [12]. Unofficial ROS and ROS 2 interfaces have been developed to bridge the SDK with modern middleware. For example, `unitree_ros2` provides an LCM-based ROS 2 interface for A1 [11]. The official software guides detail high- and low-level control modes, joint limits, and safety features such as fall and disconnection protection [10], which are relevant when integrating external perception modules.

Langdun and Oswald built a vision-based AI demonstrator using a Unitree A1 robot that responds to four predefined hand gestures captured by the onboard camera [9]. Their pipeline uses MediaPipe Hands for 2D hand keypoints, a simple machine-learning classifier (logistic regression) for gesture recognition, and high-level commands sent via the Unitree SDK, with all computation running on an external laptop connected over Wi-Fi. Their work inspired our HRI scenario but differs in several key aspects: we focus on full-body pose rather than hand-only gestures, run inference on an embedded Jetson Orin Nano mounted on the robot, and integrate the perception stack into a ROS 2 system designed to feed a separate gesture recognition module.

D. Hardware and software platforms

NVIDIA Jetson platforms, including Jetson Orin Nano, are supported by the CUDA Toolkit and JetPack SDK, which bundle CUDA, cuDNN, TensorRT, and related tools for GPU-accelerated AI [15]–[17]. Our project uses JetPack 6 with CUDA 12.6 on Ubuntu 22.04 running ROS 2 Humble. For cameras, we rely on the `v4l2_camera` driver for USB webcams [14], and we considered Intel RealSense cameras via the `librealsense` SDK [13], although legacy software versions on the robot side made RealSense integration impractical in our timeframe.

Beyond the technical aspects, broader discussions on AI and autonomy emphasize that AI capabilities and deployment autonomy should be treated as separate dimensions [8]. Our system is a narrow, task-specific perception module that operates in a low-autonomy regime: it only provides pose data and does not directly decide robot actions, keeping humans in control of high-level interaction design.

III. METHODOLOGY

A. System overview

The overall system architecture involves three main components:

- 1) **Perception pipeline** running on the Jetson Orin Nano mounted on the Unitree A1. It captures RGB images from a USB webcam, runs a YOLO11m-pose model to detect humans and estimate 2D body keypoints, and publishes the results and basic performance statistics as ROS 2 messages.

- 2) **Gesture recognition module** developed by a partner team. It subscribes to our pose messages, interprets the skeletal data, and classifies human gestures or states.
- 3) **Robot control layer** using an LCM-based bridge and Unitree high-level commands to send body-level velocity and posture commands to the A1 controller over Ethernet.

Our work covers the first component and the interface between perception and the gesture module. We do not design or train the gesture recognizer itself. The perception pipeline is designed to be agnostic to the specific gesture representation: it simply provides, for each detected person, a set of 17 body keypoints in image coordinates and their confidence scores, along with an identifier and timing information.

B. Hardware platform

1) *Unitree A1 robot*: Unitree A1 is a torque-controlled quadruped robot with 12 actuated joints (3 per leg: hip, thigh, calf) and an onboard motion controller that runs real-time control loops at up to 1000 Hz [10]. The official software guide specifies joint limits, link lengths, and built-in safety mechanisms such as fall protection, overtemperature shutdown, and disconnection protection for high- and low-level control modes [10]. High-level control allows the user to command body-level quantities such as forward, lateral, and rotational speeds, body height, and body orientation (yaw, pitch, roll) in normalized ranges, which are mapped to physical velocities and postures by the internal gait controller.

In our project, we treat A1 as a black-box locomotion platform: the perception system does not directly interact with joint-level control. Instead, high-level commands are sent by the gesture module and control layer via LCM channels understood by the Unitree control stack.

2) *Jetson Orin Nano and camera*: We use an NVIDIA Jetson Orin Nano 4 GB module [17], mounted on the back of the A1 robot and powered by an independent power supply to avoid draining the robot's battery. The Jetson is connected to the A1 controller via Ethernet for robot control and communicates with a development laptop over a separate network when needed. The operating system is Ubuntu 22.04 with JetPack 6, providing CUDA 12.6, cuDNN, and other components required for GPU-accelerated inference [15], [16].

For visual input we use a standard USB webcam connected directly to the Jetson rather than the built-in RealSense camera. While the A1 platform is equipped with a RealSense depth camera, its associated software stack on the robot was based on older SDK versions and ROS 1 integration, which made it cumbersome to access from our ROS 2-based pipeline within the course timeframe. In contrast, a generic UVC webcam combined with the `v4l2_camera` package [14] provides a simple and robust source of RGB images.

C. Software stack and containerization

1) *Base system and containers*: Our software stack is deployed using Docker both on development laptops and on the Jetson Orin Nano. On the development side, we

use a custom amd64 image `fabook/fox:v0.0.1` based on `ros:humble-ros-base`. This image installs ROS 2 Humble, a set of perception and visualization packages (including `rviz2`, `v4l2_camera`, and `image-transport`), and Python dependencies such as ONNX Runtime 1.18.1. The container is launched via `docker-compose` with host networking, X11 forwarding, and access to local video devices, allowing us to run RViz and camera drivers directly from inside the container.

On the Jetson Orin Nano we use a separate arm64 image `fabook/fox:jetson-v0.0.3` based on the Ultralytics `ultralytics:latest-jetson-jetpack6` image, which already includes JetPack 6 (CUDA, cuDNN, TensorRT, L4T). On top of this base we install ROS 2 Humble Desktop and the same set of perception packages as in the development image. Both images create a non-root user and a ROS 2 workspace at `/home/mobile/ros2_ws`, where our packages are mounted from the host (`project_fabian` repository) using bind volumes. The containers are started with `network_mode: host`, access to `/dev/video*` and `/dev/dri`, and GPU access enabled via `gpus: all` and `NVIDIA_VISIBLE_DEVICES=all`.

2) *ROS 2 packages*: The main ROS 2 packages in our system are:

- `f_bringup` — launch files that start the camera driver, perception node, RViz, and TF broadcasters.
- `f_interfaces` — custom message definitions used to exchange pose data (`PersonBodyArray`), actions (`PersonAction`), and high-level commands (`FoxCommand`).
- `f_human_detection2` — the core perception node that subscribes to RGB images, runs the YOLO11 pose model, and publishes pose messages and metrics.
- `f_gesture_recognition` — the gesture recognition package maintained by the partner team; it subscribes to our pose messages and outputs classified actions.
- `f_fox_command` — converts gesture labels into normalized command strings for the robot (e.g., "walking", "salute", "jumping").
- `f_a1_lcm_control` — an LCM-based bridge that converts high-level commands into Unitree `HighCmd` messages and sends them to the A1 controller.

On both development laptops and the Jetson, we use the `v4l2_camera_node` to publish raw images from UVC webcams on the `/image_raw` topic [14].

D. Perception pipeline

1) *Camera abstraction*: The camera abstraction layer is designed to make the perception pipeline independent of the physical camera. During development, the input comes from a USB webcam on a laptop or directly on the Jetson. In principle, the same pipeline could be fed from a RealSense camera via the `librealsense` ROS wrapper [13], but we did not reach a stable integration with the robot's legacy RealSense setup.

The camera node publishes:

- `/image_raw` — RGB frames as `sensor_msgs/Image`.
- Camera info topics (optional) that include intrinsic parameters.

The perception node is configured to subscribe to this topic regardless of the underlying hardware, simplifying testing.

2) *Detection and pose estimation model:* Our pose estimation backbone is an off-the-shelf YOLO11 pose model provided by Ultralytics. We empirically compared three variants (`s`, `m`, and `l`) exported to ONNX and executed on the Jetson Orin Nano (see Section IV-C). The `s` variant provides the highest throughput but lower pose fidelity, while the `l` variant yields the best pose quality but incurs higher latency and lower FPS. Based on this comparison, we selected the `m` variant (YOLO11m-pose) as a compromise between model size, accuracy, and runtime performance on our 4 GB configuration.

The model predicts, for each detected person, a bounding box and a set of 17 2D keypoints corresponding to standard body joints (e.g., shoulders, elbows, wrists, hips, knees, ankles). The model is used as published, without additional fine-tuning on our own data. This choice reflects the educational nature of the project and time constraints: our focus is on deployment and integration rather than model training.

For deployment on the Jetson, the model is exported to ONNX and executed via ONNX Runtime inside the container. We use ONNX Runtime 1.18.1 configured to prefer the `CUDAExecutionProvider` when available and fall back to CPU otherwise. During development we encountered version constraints: a specific ONNX Runtime version was required, and the exported model had to use an appropriate opset version. Resolving these constraints and configuring the container for GPU access was one of the practical challenges of the project.

We also experimented with TensorRT-based deployment at a prototype level. Using the YOLO11m-pose checkpoint, we attempted to export the model to ONNX and to build TensorRT engines with reduced precision (FP16/INT8) to explore potential speedups. These experiments helped us understand the workflow of TensorRT engine building and quantization conceptually. However, we did not succeed in obtaining a stable TensorRT-based pipeline that worked end-to-end inside our ROS 2 Docker environment on JetPack 6, mainly due to version incompatibilities between TensorRT, ONNX Runtime, JetPack, and Python bindings and the added complexity of maintaining such a stack. Consequently, the final deployed system relies entirely on ONNX Runtime.

3) *ROS 2 node functionality:* The `f_human_detection2` node performs the following steps:

- 1) Subscribes to `/image_raw` and converts incoming images to the format expected by the model (resizing to 640×384 , normalization and letterboxing).
- 2) Runs the YOLO11m-pose model via ONNX Runtime on the Jetson GPU.
- 3) Parses the model output to extract, for each detected person:

- a bounding box with class label (person) and confidence score;
- a set of 17 2D keypoints represented as (x, y, score) triplets in image coordinates.

4) Applies a simple tracker (`SimpleByteLike`) that assigns persistent IDs to detections based on IoU matching of bounding boxes across frames.

5) Publishes the resulting skeletons and detection scores on a custom topic defined in `f_interfaces`, along with basic performance statistics such as current FPS and processing latency.

For visual debugging we define RViz configurations that display the input image along with overlaid bounding boxes, keypoints, and skeleton links, color-coded by person ID. This was essential for verifying pose quality in different scenes and for ensuring that the perception node did not flood the system with spurious detections.

E. Interface to gesture recognition and robot control

The custom messages defined in `f_interfaces` implement the following data flow:

- **Pose output:** `PersonBodyArray` contains a header and a list of `PersonBody` instances. Each `PersonBody` carries a track identifier, a detection score, a bounding box in pixels, and a flattened array of 17 keypoints encoded as (x, y, score) triplets.
- **Gesture recognition:** the `f_gesture_recognition` package subscribes to `PersonBodyArray`, computes temporal features, and outputs `PersonAction` messages containing high-level labels and confidence scores.
- **Command mapping:** the `f_fox_command` package converts `PersonAction` messages into `FoxCommand` messages with normalized command strings suitable for robot control (e.g., "walking", "salute", "jumping").
- **Robot control:** the `f_al_lcm_control` package subscribes to `FoxCommand` and uses a Python LCM client to publish serialized `Unitree HighCmd` structures on the `LCM_High_Cmd` channel at a fixed rate (e.g., 100 Hz). The node implements simple motion patterns for each command and falls back to a standing mode if no valid commands are received for more than a timeout window (e.g., 1.5 s).

This design decouples perception and gesture recognition from the low-level locomotion controller while still enabling intuitive gesture-based control of the quadruped.

IV. EXPERIMENTS AND EVALUATION

The experimental evaluation focuses on basic performance characteristics of the perception pipeline, as we did not perform formal accuracy benchmarking against labelled datasets. Our goal is to validate that the system achieves real-time operation on Jetson Orin Nano and provides stable skeletons suitable for downstream gesture recognition.

TABLE I
RUNTIME COMPARISON OF YOLO11 POSE VARIANTS ON JETSON ORIN NANO (ONNX, RVIZ OFF, 640×384).

Model variant	FPS	Latency [ms]
YOLO11s (ONNX)	22–23	30–40
YOLO11m (ONNX)	13–14	60–70
YOLO11l (ONNX)	10–11	80–90

A. Experimental setup

All measurements were performed on the Jetson Orin Nano 4 GB module running Ubuntu 22.04 with JetPack 6 and CUDA 12.6, inside the Docker container described above. The camera input is a USB webcam publishing RGB images that are resized to 640×384 for inference. The perception container runs the `f_human_detection2` node.

We tested typical HRI scenarios with one person standing in front of the robot or camera at distances of roughly 1–2 m and performing simple movements (raising arms, turning, walking). Multi-person scenes were also inspected qualitatively, but the primary focus was on a single interaction partner, as this matches the intended use case for gesture-based commands.

B. Latency and throughput

When measuring the pose node in isolation (without RViz and additional processing), the YOLO11m-pose pipeline achieves approximately 15 FPS on the Jetson Orin Nano at an input resolution of 640×384 , including model inference and ROS 2 communication overhead. In the full end-to-end configuration with RViz visualization and all auxiliary nodes enabled, the effective frame rate is closer to 15 FPS, which we report as the characteristic performance of the deployed system. In both cases, the measured end-to-end latency between capturing a raw image and publishing the corresponding pose message is around 100 ms.

These values are based on internal timing instrumentation in the perception node (using an exponentially smoothed FPS estimator) and visual inspection of RViz updates; we did not perform more detailed profiling with tools such as `tegrastats` or `Nsight`. From a practical standpoint, the observed latency and frame rate are sufficient for smooth visual feedback and responsive gesture recognition in our educational scenarios. We did not observe noticeable stuttering or lag in the RViz visualization or in the downstream gesture predictions when integrated with the partner team’s module.

C. Model comparison: YOLO11s/m/l

To quantify the trade-offs between different model sizes, we benchmarked three YOLO11 pose variants (s, m, and l) exported to ONNX and executed on the Jetson Orin Nano with RViz disabled. All measurements were taken at an input resolution of 640×384 under the same conditions as in the previous subsection. The results are summarized in Table I.

The smallest model, YOLO11s, achieves the highest throughput (approximately 22–23 FPS) and the lowest end-to-end latency (30–40 ms), but at the cost of reduced pose accuracy, especially for smaller joints and more challenging

poses. The largest model, YOLO11l, offers the best pose fidelity but runs at only 10–11 FPS with 80–90 ms latency, which leaves less headroom for additional processing in a complete robotic stack.

The YOLO11m variant provides an intermediate operating point, with 13–14 FPS and 60–70 ms latency in this configuration. In practice, we found this configuration to offer a satisfactory balance between runtime speed and pose stability for our human–robot interaction scenarios. Consequently, YOLO11m-pose was adopted as the default model for all subsequent experiments and integration with the gesture recognition module.

D. Qualitative pose assessment

Since we did not collect a labelled dataset for quantitative evaluation, pose quality was assessed qualitatively by visual inspection in RViz across different scenes and lighting conditions. For typical indoor lighting and moderate distances (1–3 m), the YOLO11m-pose model produces stable and plausible skeletons, with keypoints attached to the correct limbs and relatively few spurious detections. At larger distances or under challenging lighting, small joints such as ankles can become noisy or be missed, which is consistent with the behaviour of similar models reported in the literature.

When integrated with the gesture recognition module, the pose data proved robust enough to support meaningful gesture classification: the downstream module produced consistent predictions, and our node did not flood the interface with false positives or unstable skeletons. However, we did not systematically test extreme conditions such as strong backlighting, crowded scenes, or rapid motions.

V. DISCUSSION

A. Engineering challenges

Several practical challenges arose during this project:

- **Legacy robot software.** The A1 platform we used runs an older software stack on its onboard PC (legacy Linux and ROS 1), with a custom communication system and RealSense integration based on outdated SDKs. This made it infeasible to run modern CUDA-based perception directly on the onboard PC, forcing us to rely on an external embedded platform (Jetson Orin Nano) mounted on the robot. Bridging between this external platform and the A1 controller required careful setup of Ethernet networking and the use of an LCM-based communication bridge rather than a native ROS 2 interface.
- **Model deployment, ONNX Runtime, and TensorRT.** Exporting the YOLO11m-pose model to ONNX and running it with GPU acceleration inside a Jetson container required matching specific ONNX Runtime and opset versions. Misaligned versions led to runtime errors and unsupported operators. In addition, configuring the NVIDIA Container Toolkit and container images to expose the GPU correctly was non-trivial for course participants. Our attempts to integrate a TensorRT engine introduced further complexity: while the export and engine-building

steps could be partially reproduced, combining TensorRT, ONNX Runtime, JetPack 6, and Python bindings in a single ROS 2 container without version conflicts turned out to be difficult within the project timeframe, and no stable TensorRT-based pipeline was achieved.

- **Thermal and power considerations.** The Jetson Orin Nano can overheat or throttle under heavy load if not adequately cooled. In our setup it is powered directly from the A1 power bus (12 V), sharing the same battery as the robot. This simplifies wiring but imposes a stricter power budget, since the same battery must supply both locomotion and computation. Thermal behaviour remained acceptable in our tests, and we did not observe critical brownouts, but we did not perform systematic long-duration stress testing.

B. Limitations

The project has several limitations that are important to acknowledge:

- **No model training or fine-tuning.** We use an off-the-shelf YOLO11m-pose model without any fine-tuning on data collected from the A1 or our specific environment. While qualitative results are good enough for our purposes, further improvements could be obtained by fine-tuning on robot-specific viewpoints and lighting conditions.
- **Limited evaluation.** We did not conduct formal quantitative evaluation of detection or keypoint accuracy (e.g., mAP or OKS metrics) and did not perform systematic experiments under varied conditions (e.g., crowded scenes, outdoor environments, extreme lighting). Our evaluation is limited to basic FPS/latency measurements and qualitative inspection.
- **Partial integration with the robot.** At the time of writing, we are still in the process of fully integrating the perception pipeline into the robot's control loop. High-level commands are sent via the LCM bridge based on gesture recognition outputs, but the system has not undergone extensive testing in complex scenarios (e.g., dynamic locomotion around humans, safety-critical interactions).
- **No TensorRT deployment in the main pipeline.** We investigated the possibility of using TensorRT to further optimize the model for Jetson GPUs, as commonly recommended in the literature [5], [6]. We explored export and engine-building workflows and studied INT8 quantization, which suggests that substantial inference speedups may be achievable. However, due to version incompatibilities and integration complexity, we did not manage to deploy a stable TensorRT-based pipeline in our Dockerized ROS 2 environment on JetPack 6, so all reported results are based on ONNX Runtime.

C. Educational outcomes

Despite these limitations, the project fulfills its educational goals. The team gained hands-on experience with:

- deploying modern deep learning models on embedded hardware and running inference on a GPU using CUDA, ONNX Runtime 1.18.1, and JetPack 6;
- configuring CUDA and AI software stacks via JetPack and Docker, including GPU-enabled containers on Jetson;
- integrating perception with ROS 2, custom message types, and visualization tools;
- interfacing with a real quadruped robot through a legacy SDK and a UDP/LCM-based communication bridge;
- exploring TensorRT engine building and quantization workflows at a prototype level, even though the final deployed system uses ONNX Runtime.

The project also serves as a foundation for future work, such as a thesis-level project aimed at building a more sophisticated interactive robot agent that can learn and perform more complex behaviours, analogous to teaching a real dog new tricks.

VI. PROJECT SUMMARY FOR COURSE REQUIREMENTS

A. Team name, topic, and repository

The project is carried out by **105a Team #1**. The team members are:

- Evgenii Shlomov — e.shlomov@innopolis.university
- Artyom Tuzov — a.tuzov@innopolis.university
- Dmitriy Vizitei — d.vizitei@innopolis.university

The project topic is “*Robofox-human interaction with CV and deep learning*”, focusing on real-time human pose estimation on an embedded GPU and its integration with a Unitree A1 quadruped robot for gesture-based interaction.

The main code repository is:

- **Repository:** https://github.com/Innopolis-Robotics-Society/project_fabian

We treat this repository as containing the merged work of the team (previously developed on a dedicated team branch).

B. What we tried to do during the project

During the project we aimed to:

- Design a perception pipeline that performs real-time human pose detection on an embedded GPU (Jetson Orin Nano) and exposes skeletal data to a separate gesture recognition module.
- Integrate this pipeline into a ROS 2 Humble system running in GPU-enabled Docker containers on JetPack 6 with CUDA 12.6 and ONNX Runtime 1.18.1.
- Compare several YOLO11 pose model variants (s, m, l) on the Jetson and select a model that balances accuracy and runtime speed.
- Build and test a camera abstraction layer that works with generic USB webcams (and potentially RealSense) via `v4l2_camera` and `librealsense`.
- Connect the perception pipeline to a Unitree A1 robot through a combination of an LCM-based communication bridge and the Unitree high-level control interface.
- Investigate exporting models to ONNX and explore the use of TensorRT for further optimization on Jetson, while

ultimately deploying the system with ONNX Runtime due to integration constraints.

Some of these objectives were fully achieved (e.g., YOLO11m-pose deployment, ROS 2 integration, YOLO11s/m/l comparison, end-to-end command pipeline), while others remained exploratory due to time constraints (e.g., stable RealSense integration on the robot and a robust TensorRT pipeline).

C. Main results and artifacts

The main results of the project include:

- A working ROS 2 Humble perception node (`f_human_detection2`) running on Jetson Orin Nano 4 GB, performing real-time human pose estimation at approximately 15 FPS and ~ 100 ms latency at 640×384 in the full end-to-end configuration with visualization.
- GPU-enabled Docker images (amd64 and arm64) with JetPack 6 and ONNX Runtime 1.18.1, capable of running YOLO11 pose models with GPU acceleration on Jetson Orin Nano.
- A custom ROS 2 message definition (`PersonBodyArray`) for 2D skeletons, confidence scores, and performance metrics, used as the interface between perception and gesture recognition.
- An end-to-end integration with the partner team's gesture recognition module and the Unitree A1 control interface, demonstrating the full pipeline from camera images to high-level commands sent to the robot via LCM.
- Benchmark results for YOLO11s/m/l model variants on Jetson Orin Nano, providing empirical guidance on model selection for embedded deployment.

Additional artifacts like RViz screenshots, recorded rosbags, and architecture diagrams are stored alongside the code in the repository or internal project folders and can be provided to instructors as needed.

D. Timeline of the project

The project timeline can be summarized as follows:

- **September:** Brainstorming of approaches, studying Unitree A1 documentation and SDKs, and trying to understand the existing software stack on the robot. Initial exploration of possible CV models and hardware configurations.
- **October:** Design of the system architecture and ROS 2 node layout; implementation of the first ROS 2 nodes and bringup scripts; bringing the pipeline up on development laptops using USB webcams; testing initial pose models. The Docker-based ROS 2 environment with ONNX Runtime was stabilized.
- **November:** Focus on model selection and optimization. We tested multiple pose models and YOLO11 variants under different conditions, experimented with ONNX export and ONNX Runtime backends, and attempted TensorRT-based optimization. In parallel we achieved an

end-to-end integration of the perception pipeline with the gesture recognition module and the robot, verifying that poses and gestures were processed correctly in real time.

E. Individual contributions

The individual contributions of the teammates can be summarized as:

- **Evgenii Shlomov:** Focus on literature review, system-level design, and Docker infrastructure. Responsible for setting up GPU-enabled Docker images with ROS 2, JetPack 6, and ONNX Runtime, integrating Ultralytics/YOLO tooling into the containers, and helping to structure the overall perception pipeline and documentation.
- **Artyom Tuzov:** Focus on models, backends, and the perception pipeline implementation. Responsible for selecting and benchmarking YOLO11 pose variants, exporting models to ONNX, integrating ONNX Runtime into the ROS 2 node, measuring FPS/latency, and implementing the pose-processing logic (parsing outputs, publishing skeletons, and basic tracking). Also contributed to ROS 2 integration and debugging.
- **Dmitriy Vizitei:** Focus on the Unitree A1 robot, SDKs, and hardware integration. Responsible for understanding the legacy software stack on the robot, configuring the network and control interface, setting up the LCM-based communication with the A1 controller, and designing and testing the motion patterns corresponding to high-level commands. Also contributed to ROS 2 bringup and testing with real hardware.

All team members collaboratively worked on ROS 2 integration, testing, and debugging, including RViz visualization, message definitions, and end-to-end tests with the partner team's gesture recognition module.

VII. CONCLUSION

We presented the design and deployment of a real-time human pose estimation pipeline on an NVIDIA Jetson Orin Nano 4 GB module mounted on a Unitree A1 quadruped robot. Using an off-the-shelf YOLO11m-pose model running inside a Docker container with JetPack 6, CUDA 12.6, ONNX Runtime 1.18.1, and ROS 2 Humble, our system achieves approximately 15 FPS at 640×384 resolution with around 100 ms end-to-end latency in the full configuration, providing 2D body keypoints and confidence scores for downstream gesture recognition by a partner module. While our contributions are primarily engineering and educational rather than algorithmic, the project highlights the practical challenges and solutions involved in bringing state-of-the-art computer vision models to real robotic platforms with legacy constraints.

Future work includes revisiting TensorRT-based optimization when tooling and version constraints are more favourable, designing a more convenient and robust communication bridge between the Jetson and the A1 controller (e.g., via a ROS 2 interface instead of direct LCM calls), and fine-tuning the pose model on robot-specific data to increase robustness

in challenging conditions. A more comprehensive evaluation campaign, including safety and user studies, would also be required for any deployment in real-world HRI scenarios.

REFERENCES

- [1] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, “DETRs Beat YOLOs on Real-time Object Detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, arXiv:2304.08069, 2023.
- [2] H. Ouyang, “DEYOv3: DETR with YOLO for Real-time Object Detection,” *arXiv preprint arXiv:2309.11851*, 2023.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [4] D. Maji, S. Nagori, M. Mathew, and D. Poddar, “YOLO-Pose: Enhancing YOLO for Multi Person Pose Estimation Using Object Keypoint Similarity Loss,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2022, pp. 2636–2645.
- [5] M. Afifi, Y. Ali, K. Amer, M. Shaker, and M. El-Helw, “Robust Real-Time Pedestrian Detection on Embedded Devices,” in *Proc. Thirteenth Int. Conf. on Machine Vision (ICMV)*, vol. 11605, 2021, Art. no. 1160529.
- [6] H. Feng, G. Mu, S. Zhong, P. Zhang, and T. Yuan, “Benchmark Analysis of YOLO Performance on Edge Intelligence Devices,” *Cryptography*, vol. 6, no. 2, p. 16, 2022.
- [7] R. Schrijvers, S. Puttemans, T. Callemeyn, and T. Goedemé, “Real-Time Embedded Person Detection and Tracking for Shopping Behaviour Analysis,” in *Advanced Concepts for Intelligent Vision Systems (ACIVS 2020)*, Lect. Notes Comput. Sci., vol. 12002, pp. 541–553, 2020.
- [8] M. R. Morris, J. Sohl-Dickstein, N. Fiedel, T. Warkentin, A. Dafoe, A. Faust, C. Farabet, and S. Legg, “Levels of AGI for Operationalizing Progress on the Path to AGI,” *arXiv preprint arXiv:2311.02462*, 2023.
- [9] T. S. Langdun and M. Oswald, “Building a Vision-Based AI Demonstrator with Unitree A1 Quadruped Robot,” Bachelor Thesis, ZHAW School of Engineering, 2022.
- [10] Unitree Robotics, *Unitree A1 Software Guide*, Version 2.0.
- [11] M. Takahashi, “unitree_ros2: Unofficial ROS 2 Interface for Unitree A1,” GitHub repository. [Online]. Available: https://github.com/mayataka/unitree_ros2
- [12] Unitree Robotics, “unitree_legged_sdk and unitree_ros_to_real,” GitHub repositories. [Online]. Available: https://github.com/unitreerobotics/unitree_legged_sdk and https://github.com/unitreerobotics/unitree_ros_to_real
- [13] Intel Corporation, “Intel RealSense SDK 2.0 (librealsense),” GitHub repository. [Online]. Available: <https://github.com/IntelRealSense/librealsense>
- [14] Bold Hearts, “ros2_v4l2_camera: A ROS 2 Camera Driver Using Video4Linux2,” GitLab repository. [Online]. Available: https://gitlab.com/boldhearts/ros2_v4l2_camera
- [15] NVIDIA Corporation, “CUDA Toolkit Documentation 12.6 Update 3,” 2024. [Online]. Available: <https://docs.nvidia.com/cuda/archive/12.6.3/>
- [16] NVIDIA Corporation, “NVIDIA JetPack SDK Documentation,” 2025. [Online]. Available: <https://docs.nvidia.com/jetson/jetpack/index.html>
- [17] NVIDIA Corporation, “Jetson Orin Nano Developer Kit User Guide,” [Online]. Available: <https://developer.nvidia.com/embedded/learn/jetson-orin-nano-devkit-user-guide/index.html>